

AUTH SYSTEM

Complete Developer Documentation

DOCUMENT 4 OF 4

Deployment, Operations & Troubleshooting Guide

1. Deployment Overview

Deployment means taking your code from your laptop and running it on a server that real users can access over the internet. This document covers the complete path from local code to a running production application.

Budget-First Approach

This guide starts with the lowest-cost option that handles real traffic — approximately \$40 per month. Every step is designed so you can upgrade to the next level without rewriting any application code. Only the infrastructure configuration changes.

2. Phase 1 — Single Server Deployment (Best for Starting Out)

One server runs everything: the backend, frontend, database, Redis, and Nginx. This is the most cost-effective way to start and handles thousands of daily users comfortably.

2.1 Server Requirements

Requirement	Minimum	Recommended
CPU	2 vCPU	4 vCPU
RAM	4 GB	8 GB
Storage	40 GB SSD	80 GB SSD
Operating System	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
Monthly Cost	~\$20 USD	~\$40-\$80 USD
Provider options	Hetzner CX21	Hetzner CX31 / DigitalOcean Droplet

2.2 Setting Up the Server — Complete Steps

Step 1 — Create your server and connect to it:

```
# On your LOCAL laptop — connect to your new server
# Replace YOUR_SERVER_IP with the IP from your hosting provider
ssh root@YOUR_SERVER_IP

# First thing: update all system packages
apt update && apt upgrade -y

# Create a non-root user for security
adduser appuser
usermod -aG sudo appuser
usermod -aG docker appuser
```

Step 2 — Install Docker and Docker Compose:

```
# Install Docker
curl -fsSL https://get.docker.com | sh

# Install Docker Compose plugin
apt install -y docker-compose-plugin

# Verify
docker --version
docker compose version

# Start Docker automatically on server reboot
systemctl enable docker
systemctl start docker
```

Step 3 — Install Nginx and Certbot (for free HTTPS):

```
# Install Nginx
apt install -y nginx

# Install Certbot for free Let's Encrypt SSL certificates
apt install -y certbot python3-certbot-nginx

# Start Nginx
systemctl enable nginx
systemctl start nginx
```

Step 4 — Deploy your application:

```
# Switch to appuser
su - appuser

# Clone your repository
cd /opt
git clone https://github.com/YOUR_USERNAME/auth-system.git
cd auth-system

# Create the production .env file
cp backend-api/.env.example backend-api/.env
nano backend-api/.env
# Fill in ALL real production values
# Set APP_ENV=production
# Set DEBUG=false
# Set COOKIE_SECURE=true
# Set real database password
# Set real SendGrid and Twilio keys

# Start all services
docker compose up -d --build

# Run database migrations
docker compose exec backend alembic upgrade head

# Create the first admin user
docker compose exec backend python scripts/create_superuser.py \
  --email admin@yourdomain.com \
  --username admin \
  --password "YourStrongPass@123" \
  --full-name "Admin" \
  --phone "+919876543210"
```

Step 5 — Get a free HTTPS certificate:

```
# Replace yourdomain.com with your actual domain
# Your domain DNS must already point to this server IP
certbot --nginx -d yourdomain.com -d www.yourdomain.com

# Certbot will ask for your email address
# It automatically configures Nginx with HTTPS

# Test automatic certificate renewal
certbot renew --dry-run

# Certificates auto-renew every 90 days — no manual action needed
```

3. Domain Name and DNS Setup

A domain name (like yourdomain.com) makes your application accessible by name instead of IP address. You need to point the domain to your server.

3.1 Setting Up Cloudflare (Recommended — Free)

1. Buy a domain from any registrar (Namecheap, GoDaddy, Google Domains)
2. Create a free account at cloudflare.com
3. Click Add Site and enter your domain name
4. Cloudflare will scan your existing DNS records
5. Change your domain's nameservers to the Cloudflare ones shown
6. In Cloudflare DNS, add an A record pointing yourdomain.com to your server IP
7. Enable the orange cloud icon (proxy) — this activates DDoS protection and CDN

Cloudflare free tier gives you:

- Free DDoS attack protection for your application
- Free CDN — static files served from servers close to your users
- Free SSL/TLS termination — HTTPS from browser to Cloudflare
- Bot protection — reduces automated attack traffic
- Analytics — see how many visitors you have

4. Automated Deployment — CI/CD Pipeline

CI/CD stands for Continuous Integration and Continuous Deployment. It means every time you push code to GitHub, the pipeline automatically tests it and deploys it to your server. You never manually copy files to the server again.

4.1 What the Pipeline Does

Stage	Name	What It Does
1	Lint	Checks code formatting and catches obvious errors without running the code
2	Type Check	Verifies TypeScript and Python type annotations are correct
3	Test	Runs all pytest tests — deployment stops if any test fails
4	Build	Creates Docker images for backend and frontend

Stage	Name	What It Does
5	Push	Uploads Docker images to GitHub Container Registry
6	Migrate	Runs alembic upgrade head on the server before switching traffic
7	Deploy	Rolling restart — zero downtime for users
8	Notify	Sends Telegram message: deployment succeeded or failed

4.2 GitHub Secrets to Configure

Go to your GitHub repository > Settings > Secrets and variables > Actions. Add each of these:

Secret Name	What to Put There
VPS_HOST	Your server's IP address, e.g. 123.456.789.012
VPS_USER	SSH username — usually ubuntu or your chosen username
VPS_SSH_KEY	Your SSH private key — the contents of ~/.ssh/id_rsa
TELEGRAM_CHAT_ID	Your Telegram chat ID for deployment notifications
TELEGRAM_BOT_TOKEN	Your Telegram bot token from BotFather

4.3 How to Generate an SSH Key for Deployment

```
# On your LOCAL laptop — generate a dedicated deployment key
ssh-keygen -t ed25519 -C 'github-deploy' -f ~/.ssh/deploy_key

# This creates two files:
# ~/.ssh/deploy_key    (PRIVATE key — goes into GitHub Secrets)
# ~/.ssh/deploy_key.pub (PUBLIC key — goes onto your server)

# Copy the public key to your server
ssh-copy-id -i ~/.ssh/deploy_key.pub appuser@YOUR_SERVER_IP

# Print the private key to copy into GitHub Secrets
cat ~/.ssh/deploy_key
# Copy ALL of this output including the BEGIN and END lines
# Paste it as the value of VPS_SSH_KEY in GitHub Secrets
```

5. Monitoring and Observability

Monitoring means knowing what your application is doing and being alerted when something goes wrong — before users complain. Start with free tools and upgrade as traffic grows.

5.1 Free Monitoring Stack (Phase 1)

Tool	Purpose	How to Set Up
UptimeRobot	Pings your site every 5 min, emails if down	Free at uptimerobot.com — add your URL
Sentry free tier	Captures Python exceptions with full context	Add SENTRY_DSN to .env, free up to 5,000 errors/month
FastAPI /metrics	Prometheus metrics endpoint built in	Already included — access at /metrics
Docker logs	All container output	<code>docker compose logs -f backend</code>
Error emails	Support team notified on every crash	Already built in — set <code>ERROR_NOTIFICATIONS_ENABLED=true</code>

5.2 Essential Log Commands

```
# View live logs from the backend
docker compose logs -f backend

# View last 100 lines of backend logs
docker compose logs --tail=100 backend

# View logs from all services simultaneously
docker compose logs -f

# Check if all containers are running
docker compose ps

# Check server resource usage
docker stats

# Check disk space
df -h

# Check memory usage
free -m
```

6. Database Backups

Backups are non-negotiable. A database without backups is a liability, not an asset. Set these up before your first real user signs up.

6.1 Automated Daily Backup Script

```
# Create the backup script on your server
nano /opt/backup-db.sh

# Paste this content:
#!/bin/bash
DATE=$(date +%Y-%m-%d_%H-%M)
BACKUP_DIR=/opt/backups
mkdir -p $BACKUP_DIR

# Create compressed database dump
docker compose -f /opt/auth-system/docker-compose.yml \
  exec -T postgres pg_dump -U authuser authdb \
  | gzip > $BACKUP_DIR/authdb_$DATE.sql.gz

# Delete backups older than 30 days
find $BACKUP_DIR -name '*.sql.gz' -mtime +30 -delete

echo "Backup completed: authdb_$DATE.sql.gz"

# Make the script executable
chmod +x /opt/backup-db.sh

# Schedule it to run every day at 2 AM
crontab -e
# Add this line:
0 2 * * * /opt/backup-db.sh >> /var/log/db-backup.log 2>&1
```

7. Troubleshooting — Common Problems and Solutions

Problem / Error	Solution
502 Bad Gateway in browser	Backend container is down. Run: <code>docker compose up -d backend</code>
Connection refused on port 8000	Check if container is running: <code>docker compose ps</code> . Check logs: <code>docker compose logs backend</code>
cannot connect to postgres	PostgreSQL container not running or wrong DATABASE_URL. Run: <code>docker compose up -d postgres</code>
Redis connection refused	Redis container not running. Run: <code>docker compose up -d redis</code>
alembic target database not up to date	Run: <code>docker compose exec backend alembic upgrade head</code>
422 Unprocessable Entity	Request data does not match expected format. Read the detail field in the JSON response
CORS error in browser console	ALLOWED_ORIGINS in .env does not include the exact frontend URL with port
Cookies not being set	COOKIE_SECURE=true requires HTTPS. Use false for HTTP local dev. Check SameSite settings
Rate limit 429 errors	Too many requests from one IP. Wait for the window to expire or raise limits in .env
JWT token invalid error	Token expired or wrong SECRET_KEY. Check that SECRET_KEY matches between restarts
disk space full on server	Run: <code>docker image prune -a</code> and <code>docker volume prune</code> . Delete old log files
OTP not arriving via SMS	Check Twilio credentials in .env. Verify phone number format is E.164 (+country code)
Verification email not arriving	Check SendGrid API key. Look in spam folder. Verify EMAIL_FROM is a verified sender
Google OAuth state mismatch	Browser cookie expired or blocked. Ensure oauth_state cookie is allowed

8. Scaling Path — When You Outgrow One Server

These are the signals that tell you it is time to scale up, and exactly what to do at each step. No code changes are required — only infrastructure changes.

Signal / Symptom	What It Means	What to Do
CPU consistently above 70%	Server is working too hard	Add a second server, put a load balancer in front
Memory above 80%	Not enough RAM	Upgrade server size or add a

Signal / Symptom	What It Means	What to Do
		second one
DB response time above 100ms	Database is overloaded	Add PgBouncer, then move DB to managed RDS
Redis memory above 200MB	OTP/session data growing	Move to ElastiCache or larger Redis instance
Response time above 500ms	Server cannot keep up	Scale to ECS Fargate or Kubernetes
More than 100,000 users	Phase 1 infrastructure limit	Move to Phase 2: managed cloud services

9. Go-Live Checklist

Complete every item on this list before pointing your domain to the server and telling users the application is live:

	Go-Live Checklist Item
[]	Production .env file created with real credentials — APP_ENV=production
[]	DEBUG=false in production .env
[]	COOKIE_SECURE=true in production .env
[]	ERROR_NOTIFICATIONS_ENABLED=true and real SUPPORT_EMAIL set
[]	HTTPS certificate obtained and Nginx configured with TLS
[]	All services running: docker compose ps shows all as Up
[]	Database migrations applied: alembic upgrade head completed successfully
[]	Health check passes: GET https://yourdomain.com/health returns {status: ok}
[]	Readiness check passes: /readyz shows both redis and db as ok
[]	Test registration flow works end to end in a real browser
[]	Test login with password works
[]	Test OTP email is received and verification works
[]	Test OTP SMS is received and verification works
[]	Daily database backup script configured and tested
[]	UptimeRobot monitoring configured for your domain
[]	CI/CD pipeline passes all stages on the main branch
[]	Firewall allows only ports 80, 443 and SSH from your IP
[]	No .env files committed to Git — git status is clean

You Are Production Ready

If every item above is checked, your application is ready for real users. The architecture you have built will comfortably scale from your first user to hundreds of thousands before requiring any code changes. Document what you have built, keep the .env files secure, and monitor the error notifications email regularly.